EVALUACIÓN TÉCNICA EXTERNA

# Prueba técnica - Desarrollador de Soporte Java

Incidente N3: conciliación segura de pagos con reintentos e inconsistencias de datos

| Tiempo máximo | Modalidad | Entrega |
|---|---|---|
| **2 horas** | **Candidato externo** | **Repositorio Git** |

**Objetivo del ejercicio**

Resolver un incidente productivo acotado y dejar una corrección ejecutable, verificable y operable. Se busca observar cómo analiza evidencia, toma decisiones, implementa, prueba y deja trazabilidad técnica bajo una restricción realista de tiempo.

## 1. Contexto de negocio

**PagoClaro** procesa pagos para comercios. El equipo de operaciones detectó que, después de una degradación del proveedor externo de pagos, algunos pagos quedaron con estados inconsistentes o con movimientos contables duplicados. El incidente fue escalado a soporte N3 porque los reintentos manuales ya no son seguros: una nueva ejecución puede agravar la inconsistencia.

- Un pago confirmado debe terminar con un único débito neto por el valor del pago.
- Los movimientos contables son inmutables: no se eliminan ni se actualizan; una corrección se representa mediante un movimiento de reverso relacionado con el movimiento que corrige.
- Cuando la evidencia del proveedor es contradictoria o insuficiente, el sistema no debe adivinar el resultado: debe conservar los datos y señalar el caso para revisión manual.
- La misma solicitud de conciliación puede repetirse por error operativo o por un reintento técnico. Repetirla no debe generar efectos contables ni registros de auditoría duplicados.
- Dos solicitudes sobre el mismo pago pueden llegar casi al mismo tiempo. El resultado debe conservar consistencia.
- La operación debe dejar trazabilidad suficiente para soporte sin registrar secretos ni datos sensibles innecesarios.

## 2. Evidencia del incidente

### 2.1 Extracto de logs

```
2026-10-13 09:41:22.104 INFO  paymentRef=PAY-BX17 providerCall=APPROVE started attempt=1
2026-10-13 09:41:23.983 WARN  paymentRef=PAY-BX17 providerCall=APPROVE timeout after 1800ms
2026-10-13 09:41:24.106 INFO  paymentRef=PAY-BX17 retry scheduled reason=provider-timeout
2026-10-13 09:41:25.317 INFO  paymentRef=PAY-BX17 providerEvent=EVT-B2 status=APPROVED
received
2026-10-13 09:41:25.390 INFO  paymentRef=PAY-BX17 ledgerEntry=5003 type=DEBIT created
2026-10-13 09:41:25.611 WARN  paymentRef=PAY-BX17 paymentStatus already CONFIRMED during retry
2026-10-13 09:43:08.512 WARN  reconciliation job found netDebit=178000 expected=89000
paymentRef=PAY-BX17
```

*El extracto es evidencia parcial. Diferencie hechos observados, hipótesis y conclusiones.*

### 2.2 Modelo de datos disponible

```
PAYMENT(
  id NUMBER PRIMARY KEY,
  external_ref VARCHAR2(40) UNIQUE NOT NULL,
  status VARCHAR2(20) NOT NULL,            -- PENDING | CONFIRMED | REJECTED
  amount NUMBER(18,2) NOT NULL,
  updated_at TIMESTAMP NOT NULL,
  row_version NUMBER DEFAULT 0 NOT NULL
)

PROVIDER_EVENT(
  id NUMBER PRIMARY KEY,
  payment_id NUMBER NOT NULL REFERENCES PAYMENT(id),
  provider_event_id VARCHAR2(60) UNIQUE NOT NULL,
  provider_status VARCHAR2(20) NOT NULL,   -- APPROVED | DECLINED
  received_at TIMESTAMP NOT NULL
)

LEDGER_ENTRY(
  id NUMBER PRIMARY KEY,
  payment_id NUMBER NOT NULL REFERENCES PAYMENT(id),
  entry_type VARCHAR2(20) NOT NULL,        -- DEBIT | REVERSAL
  amount NUMBER(18,2) NOT NULL,
  related_entry_id NUMBER NULL REFERENCES LEDGER_ENTRY(id),
  created_at TIMESTAMP NOT NULL
)

SUPPORT_AUDIT(
  id NUMBER PRIMARY KEY,
  payment_id NUMBER NOT NULL REFERENCES PAYMENT(id),
  action VARCHAR2(40) NOT NULL,
  reason VARCHAR2(400) NOT NULL,
  created_at TIMESTAMP NOT NULL
)
```

### 2.3 Datos de muestra

| Referencia | Estado | Monto | Eventos proveedor | Ledger | Observación |
|---|---|---|---|---|---|
| PAY-AX91 | PENDING | 125000 | 1 APPROVED | 1 DEBIT | Estado del pago no coincide con la evidencia externa |
| PAY-BX17 | CONFIRMED | 89000 | 2 APPROVED | 2 DEBIT | Débito neto duplicado tras reintento |
| PAY-CQ22 | PENDING | 54000 | 1 APPROVED + 1 DECLINED | 0 | Evidencia externa contradictoria |
| PAY-DK44 | CONFIRMED | 31000 | 1 APPROVED | 1 DEBIT | Caso sano de control |

## 3. Reto

**Implemente una solución mínima en Java con Spring Boot** que permita a soporte N3 conciliar un pago por su referencia externa a partir de la evidencia disponible en base de datos. La solución debe ser ejecutable o, si usa dependencias externas que no estén disponibles, debe dejar claramente documentado cómo ejecutarla y validar sus partes principales.

### 3.1 Comportamiento funcional esperado

1. Exponer una operación de soporte que reciba la referencia externa de un pago y determine su situación usando PAYMENT, PROVIDER_EVENT y LEDGER_ENTRY.
2. Para casos con evidencia suficiente, aplicar únicamente las correcciones necesarias para dejar el pago consistente con las reglas de negocio. Los movimientos contables existentes no se borran ni se modifican.
3. Cuando exista evidencia contradictoria o no sea posible decidir con seguridad, no aplicar cambios destructivos ni compensaciones; devolver un resultado que indique revisión manual y el motivo.
4. Una repetición de la misma conciliación sobre un caso ya resuelto no debe generar nuevos efectos equivalentes.
5. Dos ejecuciones cercanas sobre el mismo pago no deben dejar el ledger o el estado del pago en una situación inválida.
6. Registrar información útil para diagnóstico y auditoría, evitando exponer información sensible o detalles internos innecesarios en la respuesta de la API.

### 3.2 Artefactos de base de datos

- Entregue una consulta SQL que permita detectar, para las últimas 24 horas, pagos con una diferencia entre el monto esperado y el débito neto, o con estado incompatible con la evidencia del proveedor. La consulta debe ser legible y justificable.
- Entregue un procedimiento o bloque PL/SQL invocable que realice de forma atómica una acción de conciliación segura para un pago, o que registre que el caso requiere revisión. Defina la interfaz que considere adecuada y documente las condiciones bajo las cuales modifica datos.
- Si propone índices, restricciones o cambios al esquema, entréguelos como scripts separados y explique brevemente su propósito. No es obligatorio modificar el esquema.

### 3.3 Pruebas

- Incluya pruebas automatizadas para el comportamiento principal y al menos dos condiciones de borde o error que considere relevantes.
- Debe existir evidencia verificable de que una repetición de la operación no duplica efectos.
- Cuando una prueba dependa de Oracle o de infraestructura no disponible durante la evaluación, puede aislarla o documentar una estrategia de validación reproducible.

## 4. Análisis y decisiones

**No se espera certeza artificial**

El material del incidente contiene vacíos deliberados. Si una decisión depende de información no disponible, documente el supuesto, el riesgo de ese supuesto y qué comprobaría en producción antes de aplicar la corrección.

- Deje un diagnóstico breve que separe: hechos observados, hipótesis de causa, validaciones que haría y corrección propuesta.
- Registre las decisiones técnicas relevantes: contexto, alternativas consideradas, decisión, trade-offs, riesgos y cómo verificó el resultado. Puede hacerlo en README, ADRs u otro formato claro.
- Incluya una nota operativa corta para soporte: cómo identificar un caso seguro de conciliar, cuándo detenerse y qué evidencia revisar después de ejecutar la solución.

## 5. Tecnologías y restricciones

| Elemento | Condición |
|---|---|
| Backend | Java y Spring Boot |
| Persistencia | Base de datos relacional; scripts SQL y PL/SQL compatibles con Oracle |
| Control de versiones | Git con historial visible de trabajo |
| Pruebas | Framework de pruebas del ecosistema Java/Spring; el candidato puede elegir herramientas concretas |
| Dependencias | Puede usar librerías razonables. Evite introducir infraestructura adicional que no sea necesaria para demostrar la solución. |
| Tiempo | Máximo 2 horas desde el inicio de la prueba |

## 6. Entrega

- Repositorio Git con el código, scripts de base de datos, pruebas y documentación.
- Instrucciones de ejecución o validación, incluyendo supuestos sobre la base de datos.
- Diagnóstico del incidente y decisiones técnicas relevantes.
- Historial de commits que permita seguir la evolución de la solución. No se exige una cantidad específica de commits.
- Si algo queda incompleto por tiempo, deje explícito qué falta, por qué lo priorizó así y cuál sería el siguiente paso.

## 7. Uso de inteligencia artificial

Puede utilizar herramientas de IA durante la prueba. Si las usa, deje trazabilidad suficiente para comprender en qué parte las utilizó, con qué objetivo, cómo validó el resultado y qué salida corrigió, rechazó o ajustó. No incluya secretos, credenciales ni información sensible en prompts. Si decide no usar IA, indíquelo brevemente.

## 8. Sugerencia de administración del tiempo

| Tramo | Enfoque sugerido |
|---|---|
| 0-20 min | Leer evidencia, identificar riesgos, decidir el alcance y registrar supuestos. |
| 20-85 min | Implementar el flujo principal y persistencia. |
| 85-110 min | SQL/PLSQL, pruebas y casos de borde. |
| 110-120 min | README, decisiones, revisión final y commits. |

*La distribución es orientativa; no forma parte de la calificación.*

## 9. Criterio de completitud visible

[ ] Existe una operación Spring Boot que permite diagnosticar/conciliar un pago por referencia externa.
[ ] Los casos de datos contradictorios se manejan sin aplicar una corrección insegura.
[ ] Repetir la operación no duplica efectos equivalentes.
[ ] Se entregan SQL y PL/SQL solicitados.
[ ] Existen pruebas automatizadas útiles.
[ ] La entrega explica diagnóstico, supuestos, decisiones, riesgos y validación.
[ ] El repositorio tiene historial Git y puede ser revisado por otra persona.